

Assignment 3: Semantic Analyzer for MiniRust

Version: v1.0

1. OBJECTIVES

- Understand static semantic analysis, symbol table management, and scope rules.
 - Implement type checking and type inference algorithms for a block-structured language.
 - Detect and report semantic errors such as redeclarations, undeclared identifiers/functions/structs, type mismatches in statements and expressions, loop control errors, and assignments to immutable bindings.
 - Develop a Python unit testing suite to verify the correctness of the semantic checker.
-

2. REQUIRED TASKS TO COMPLETE

- **Task 1: Study the AST Node definitions**
 - Re-read `src/Utils/AST.py` to ensure you understand all the abstract syntax tree representations.
 - **Task 2: Implement the StaticChecker.py file**
 - Complete the class `StaticChecker` in `src/checker/StaticChecker.py`.
 - Walk the AST to perform scope checking, type resolution, type checking, and mutability analysis.
 - Raise the appropriate semantic exceptions upon detecting any violation.
 - **Task 3: Write 100 semantic checker tests**
 - Create 100 test cases in `test/test-checker.py` (`test_1` to `test_100`) using the provided test harness.
-

3. SEMANTIC ERROR & EXCEPTION TYPES

The static semantic checker must detect violations of MiniRust type and scope rules and raise the corresponding exception classes. The exceptions are:

1. **Redeclared(kind, identifier) :**
 - Raised when an entity is declared multiple times in the same scope.
 - `kind` : one of `Variable` , `Function` , `Struct` , `Parameter` , `Member` .
 - `identifier` : name of the offending identifier.
2. **UndeclaredIdentifier(identifier) :**
 - Raised when a variable or parameter is referenced before being declared.
3. **UndeclaredFunction(function_name) :**
 - Raised when a function is called but has not been declared.
4. **UndeclaredStruct(struct_name) :**
 - Raised when a struct type is used but has not been declared.
5. **TypeCannotBeInferred(ctx) :**
 - Raised when a local variable's type cannot be determined from its initializer or context.
 - `ctx` : the AST node where the inference failure was detected.
6. **TypeMismatchInStatement(statement) :**

- Raised when statement type rules (e.g. conditions must be `bool`, return values must match function signature) are violated.
 - `statement` : the AST statement node where the mismatch occurs.
7. **TypeMismatchInExpression(expression)** :
- Raised when operand types do not match operator requirements, function arguments mismatch parameter types, or struct initializations do not match structural definitions.
 - `expression` : the AST expression node where the mismatch occurs.
8. **MustInLoop(statement)** :
- Raised when `break` or `continue` is used outside the body of a `while` or `for` loop.
9. **CannotAssignToConstant(expression)** :
- Raised when an assignment is attempted on an immutable variable, a function parameter (which is implicitly immutable in MiniRust), or a field of an immutable variable.
 - `expression` : the left-hand side expression of the invalid assignment.
-

4. DETAILED SEMANTIC CONSTRAINTS

4.1 Scope & Redeclaration Rules

- **Global Scope:**
 - Struct definitions and function definitions share distinct namespaces in global scope (i.e. you can have both `struct Point` and `fn Point()`).
 - No duplicate definitions are allowed within the same namespace (e.g. no function overloading, no duplicate struct names).
 - **Mutual Visibility (Entire Program Scope):** Struct and function definitions are visible throughout the entire program. A struct or function can reference/call other structs or functions defined after it in the source file (enabling mutually recursive functions and forward referencing of struct types without requiring forward declarations).
- **Function Scope:**
 - Parameter names must be unique.
 - **No shadowing of parameters:** You cannot declare a local variable with the same name as a function parameter anywhere inside the function body (including nested blocks). Redeclaring a parameter name as a variable raises `Redeclared(Variable, <name>)`.
- **Local Scope (Blocks):**
 - Variables declared in the same block `{ }` must have unique names.
 - Shadowing is allowed between different nested blocks (i.e., a variable in an inner block can shadow a variable in an outer block, provided it does not clash with a function parameter).

4.2 Built-in standard library

The compiler must implicitly register the following built-in functions:

- `fn print_i32(val: i32) -> void`
- `fn println_i32(val: i32) -> void`
- `fn print_f32(val: f32) -> void`
- `fn println_f32(val: f32) -> void`
- `fn print_bool(val: bool) -> void`
- `fn println_bool(val: bool) -> void`
- `fn print_string(val: string) -> void`
- `fn println_string(val: string) -> void`

- `fn println() -> void`

Every MiniRust program must define a `main` function with signature `fn main() -> void` taking no parameters.

4.3 Type Inference System

- Variable declarations without explicit type annotations (e.g. `let x = expression;`) infer their type directly from the bottom-up type of the initializer.
- Uninitialized declarations (e.g. `let mut x: Type;`) must have an explicit type. If the type is missing or cannot be resolved, raise `TypeCannotBeInferred`.

4.4 Type Matching & Coercion Rules

- **Type Coercion:** For all implicit type coercion behaviors (such as coercion from `i32` to `f32` during assignments, arithmetic operations, function argument passing, and return values), **refer directly to the MiniRust Language Specification (specifically SECTION 8: Expressions - Type Coercion)**. Do not implement any coercion rules not explicitly documented there; all other cases require exact type matching.
- **Assignments:** The type of the right-hand side expression must match the type of the left-hand side expression (subject to type coercion rules).
- **Arithmetic Operators (`+`, `-`, `*`, `/`, `%`):**
 - Operands must be compatible numeric types as defined by the language specification.
 - Modulus `%` requires `i32` operands and returns `i32`.
- **Comparisons (`==`, `!=`, `<`, `<=`, `>`, `>=`):**
 - Operands must be comparable numeric types, returning `bool`.
- **Logical Operators (`&&`, `||`, `!`):**
 - Operands must be `bool`. The result is `bool`.
- **Arrays:**
 - Array cell access `arr[idx]` requires `arr` to be an array type and `idx` to be an `i32`.
- **Structures:**
 - Field access `obj.field` requires `obj` to be of a user-defined struct type, and the struct must define a member named `field`.
 - Struct initialization must specify exactly the fields declared in the struct definition with matching types.
 - **No Recursive Structs:** A struct cannot contain a field of its own type directly or indirectly (e.g., `struct Node { next: Node }` is invalid). A recursive field declaration raises a `TypeMismatchInStatement` exception on the offending field declaration statement. (Note: Recursive and mutually recursive functions are fully allowed and do not raise any error).
- **Control Flow:**
 - Conditions for `if` and `while` must evaluate to `bool`.
 - Switch statement controlling expression must evaluate to `i32` or `bool`. Case literals must match the type of the controlling expression.

4.5 Mutability Rules

- Variables declared with `let` are immutable. They cannot be reassigned.
- Variables declared with `let mut` are mutable and can be reassigned.
- Function parameters are passed by value and behave as immutable local variables.

- Assigning to an immutable variable, a parameter, or a member of an immutable struct variable raises `CannotAssignToConstant` .

5. REQUISITES & SETUP

- Execute your unit tests: `python src/run.py test-checker`

6. PROJECT STRUCTURE

```
root/
├── src/
│   ├── grammar/
│   │   ├── MiniRust.g4      # Student's ANTLR4 grammar file
│   │   └── (generated files)
│   ├── astgen/
│   │   └── ASTGenerator.py  # Student's AST generator visitor
│   ├── checker/
│   │   ├── StaticChecker.py # Student's semantic analyzer (Assignment 3)
│   │   └── StaticError.py   # Semantic error definitions
│   └── utils/
│       ├── AST.py           # AST node definitions
│       ├── Visitor.py       # Base visitor class for AST traversal (provided)
│       └── lexerror.py      # Lexer error utilities
├── test/
│   └── test-checker.py      # Student's 100 semantic unit tests
└── requirements.txt
```

7. EVALUATION CRITERIA & GRADING

- Semantic Analysis Correctness (100%):** Grade is calculated based on passing the automated test cases run against your `StaticChecker.py` implementation.

Test Distribution Table

Category	Tests	Note
Scope Errors	test_1 – test_20	undeclared/redeclared identifiers, missing struct definitions
Type Mismatch Errors	test_21 – test_35	including coercion failures
Assignment Errors	test_36 – test_45	immutable assignment, L-Value checks
Struct Instantiation Errors	test_46 – test_57	missing/extra fields, field type mismatches
Field Access Errors	test_58 – test_65	no such field, incorrect struct

Loop Control Errors	test_66 – test_70	break/continue outside loop
Switch Statement Errors	test_71 – test_80	invalid expression type, duplicate cases, invalid case values
Function Call Errors	test_81 – test_88	incorrect arity, type mismatches
Other Semantic Errors	test_89 – test_100	return mismatches, type inference failures, main function checks
Total	100	

- **Test Cases Penalty:** Writing at least 100 unit tests in `test/test-checker.py` is mandatory. A penalty of 10% will be applied if the test file is missing or contains fewer than 100 test cases.

8. DELIVERABLES

Students must submit:

- File `src/checker/StaticChecker.py` containing the semantic checker implementation.
- File `test/test-checker.py` containing exactly 100 semantic analyzer tests.

9. DEADLINE

- **Submission Deadline:** Refer to the [deadlines.json](#) Gist file for the exact due date for Assignment 3.

10. CHANGELOG

- First release under v1.0.